patapotea

Team Onboarding Guide

Everything your team needs to get started: what to install, who works on what, how to use Git, and what to do when things go wrong.

Strathmore University · Lost & Found System · November 2026

Tech stack	React, TypeScript, Tailwind CSS (frontend) / Flask, PostgreSQL (backend)
Shared branch	dev — pull this first. All the starter code is already here.
Team	5 people: 3 frontend, 2 backend
Deadline	2 December 2026

1 — What to Install

Everyone (all 5 team members)

Install these three tools on your computer before you do anything else.

Git	Saves and tracks your code changes	git-scm.com
VS Code	Code editor (recommended)	code.visualstudio.com
Docker Desktop	Runs the database on your laptop	docker.com/products/docker-desktop

Frontend team — Persons A, B and C

Also install **Node.js v18 or higher** from nodejs.org. This gives you the npm tool you need to run the app.

Then run these commands once after you pull the code:

```
# Go into the frontend folder
cd frontend

# Download all the packages the app needs
npm install
```

```
# Make a copy of the settings file
cp .env.example .env

# Start the app — it opens at http://localhost:5173
npm run dev
```

> VS Code extensions to install: ESLint · Prettier · Tailwind CSS IntelliSense · ES7+ React Snippets

Backend team — Persons 1 and 2

Also install **Python 3.11 or higher** from python.org.

Then run these commands once after you pull the code:

```
# Go into the backend folder
cd backend

# Create a private Python environment for this project
python -m venv venv

# Turn the environment on
source venv/bin/activate # Mac and Linux
venv\Scripts\activate # Windows

# Install the Python packages
pip install -r requirements.txt

# Make a copy of the settings file
cp .env.example .env

# Start the database (run this from the ROOT folder, not inside backend/)
docker-compose up -d

# Set up the database tables (first time only)
flask db init
flask db migrate -m 'initial'
flask db upgrade

# Start the server — it runs at http://localhost:5000
python run.py
```

2 — Who Does What

Every file in the project belongs to one person. Do not edit someone else's files without asking them first. This is the main way to avoid conflicts.

Frontend — Person A

You build everything a visitor sees before they log in.

- `frontend/src/pages/public/Landing.tsx` — the home page
- `frontend/src/pages/public/Browse.tsx` — the list of found items
- `frontend/src/pages/public/ItemDetail.tsx` — the page for one item
- `frontend/src/pages/public/HowItWorks.tsx` — the how-it-works explainer
- `frontend/src/pages/auth/SignIn.tsx` — the sign-in form
- `frontend/src/pages/auth/CreateAccount.tsx` — the register form
- `frontend/src/components/layout/Navbar.tsx` — the top navigation bar
- `frontend/src/components/layout/Footer.tsx` — the page footer

Frontend — Person B

You build the pages a student uses after they log in.

- `frontend/src/pages/student/MyClaims.tsx` — the student's list of claims
- `frontend/src/pages/student/ClaimForm.tsx` — the form to claim an item
- `frontend/src/pages/student/ReportLoss.tsx` — the form to report a lost item
- `frontend/src/pages/student/Account.tsx` — the student account page
- `frontend/src/components/items/ItemCard.tsx` — the card shown in the item grid
- `frontend/src/components/claims/ClaimStepper.tsx` — the step tracker (Submitted → Collected)

Frontend — Person C

You build the guard desk pages and the admin panel.

- `frontend/src/pages/guard/GuardDashboard.tsx` — guard home with daily stats
- `frontend/src/pages/guard/LogItem.tsx` — form to log a found item
- `frontend/src/pages/guard/ItemsList.tsx` — list of all items in the store
- `frontend/src/pages/guard/ClaimsList.tsx` — list of all claims
- `frontend/src/pages/guard/VerifyClaim.tsx` — page to approve or reject a claim
- `frontend/src/pages/guard/Handover.tsx` — page to record ID check and signature
- `frontend/src/pages/admin/AdminDashboard.tsx` — admin overview with charts
- `frontend/src/pages/admin/Guards.tsx` — manage guard accounts
- `frontend/src/pages/admin/ItemsAndClaims.tsx` — view all items and claims
- `frontend/src/pages/admin/AuditLog.tsx` — read-only history of all actions
- `frontend/src/components/layout/GuardSidebar.tsx` — guard workspace side menu
- `frontend/src/components/layout/AdminSidebar.tsx` — admin workspace side menu

Backend — Person 1

You handle login, accounts, and the admin section. You also build the AuditLog model that Person 2 needs to use.

- `backend/app/__init__.py` — starts the Flask app
- `backend/app/config.py` — settings (database URL, secret keys)
- `backend/app/extensions.py` — shared tools (database, JWT, CORS)
- `backend/app/models/user.py` — the User table (students, guards, admins)
- `backend/app/models/audit_log.py` — the read-only history table
- `backend/app/routes/auth.py` — login, register, get current user
- `backend/app/routes/admin.py` — guard management, dashboard stats, audit log

Backend — Person 2

You handle the full journey of a lost item: from being logged in by a guard, to a claim being submitted, approved, and finally collected.

- `backend/app/models/item.py` — the Item table (found items)
- `backend/app/models/claim.py` — the Claim table
- `backend/app/models/handover.py` — the Handover table (ID check + signature)
- `backend/app/routes/items.py` — browse items, log a new item, update status
- `backend/app/routes/claims.py` — submit a claim, review claims, approve/reject
- `backend/app/routes/handover.py` — record a completed handover
- `backend/app/utils/helpers.py` — file upload and reference number helpers

> Shared files that everyone touches: frontend/src/types/index.ts and frontend/src/App.tsx. Talk to the team before changing these.

3 — Git Step by Step

How branches work

Think of branches like separate copies of the code. The **dev** branch is the shared copy — all the starter code is there. You create your own branch from dev, do your work there, and then send it back to dev through a Pull Request. The **main** branch is only for the final working version.

Step 1 — Get the code (do this once)

```
# Download the project to your computer

git clone https://github.com//PataPotea.git

cd PataPotea

# Switch to the dev branch

git checkout dev

git pull origin dev

# Create your own branch from dev

# Use a name that describes your work:

git checkout -b feature/frontend-public-pages # Person A

git checkout -b feature/frontend-student-pages # Person B

git checkout -b feature/frontend-guard-admin # Person C

git checkout -b feature/backend-auth-admin # Backend 1

git checkout -b feature/backend-items-claims # Backend 2
```

Step 2 — Your daily routine

```
# Start of day: get the latest changes from dev
git fetch origin
git rebase origin/dev

# Write your code, then save it
# Always name the specific files — do not use 'git add .'
git add frontend/src/pages/public/Landing.tsx

# Write a short description of what you did
git commit -m "feat: build the landing page hero section"

# Send your branch to GitHub
git push origin feature/frontend-public-pages
```

Step 3 — Send your work to dev

When you finish a piece of work, go to GitHub, open your branch, and click **New Pull Request**. Set the target branch to **dev**. Someone else on the team reviews it and merges it in.

How to switch between branches safely

If you have unfinished work and need to switch branches, save it temporarily with git stash:

```
# Save your unfinished work
git stash

# Switch to a different branch
git checkout dev

# When you come back to your branch, restore your work
git checkout feature/frontend-public-pages
git stash pop
```

4 — Merge Conflicts

What is a merge conflict?

A conflict happens when two people change the same line in the same file and Git cannot decide which version to keep. Git marks the problem in the file so you can fix it manually.

A conflict looks like this inside the file:

```
<<<<<<< HEAD
const title = 'Your version of this line'
=======
```

```
const title = 'Someone else's version'

>>>>>>> origin/dev
```

The part between <<<<<<< and ======= is your version. The part between ======= and >>>>>>> is the incoming version. Delete the markers and keep whichever version is correct (or combine both if you need to).

How to fix a conflict

```
# 1. Git will list all files with conflicts

git status

# 2. Open the file in VS Code

# You will see buttons above each conflict:

# 'Accept Current' — keep your version

# 'Accept Incoming' — keep the dev version

# 'Accept Both' — keep both (then tidy it up)

# 3. After fixing the file, mark it as resolved

git add

# 4. Tell Git to continue

git rebase --continue

# If everything goes wrong and you want to start over:

git rebase --abort
```

Rules to avoid conflicts in the first place

1	Run 'git fetch origin' and 'git rebase origin/dev' every morning before you start coding.	Keeps your branch close to dev so conflicts are small and easy to fix.
2	Only edit files that are assigned to you (see Section 2).	If everyone sticks to their own files, conflicts almost never happen.
3	Never commit directly to dev or main.	All changes must go through a Pull Request so the team can review them.
4	Tell the team before changing a shared file like types/index.ts or App.tsx.	These files are used by everyone, so changes affect the whole team.
5	If you are stuck on a conflict for more than 15 minutes, ask for help.	Never try to force-push your way out of a conflict.
6	Never use 'git push --force' on dev or main.	Force-pushing can delete other people's work permanently.

7	Open a Pull Request at the end of each day, even if the work is not finished.	Keeps everyone's branches close together and makes reviews easier.

5 — API Reference

How the frontend talks to the backend

The frontend sends requests to the backend at **http://localhost:5000/api**. This address is stored in a file called **frontend/.env** as VITE_API_URL. The file **frontend/src/services/api.ts** handles sending the login token with every request automatically, so you do not need to do that manually in each page.

POST	/auth/register	Create Account page	Anyone
POST	/auth/login	Sign In page	Anyone
GET	/auth/me	App startup	Logged in
GET	/items	Browse page	Anyone
GET	/items/:id	Item detail page	Anyone
POST	/items	Log Item page	Guard only
PATCH	/items/:id/status	Guard actions	Guard / Admin
POST	/claims	Claim Form page	Student / Staff
GET	/claims/mine	My Claims page	Student / Staff
GET	/claims/pending	Guard dashboard	Guard
GET	/claims/:id	Verify Claim page	Guard / Admin
PATCH	/claims/:id/decision	Verify Claim page	Guard
POST	/handover/:claimId	Handover page	Guard
GET	/admin/stats	Admin dashboard	Admin
GET	/admin/guards	Guards page	Admin
POST	/admin/guards	Guards page	Admin

PATCH	/admin/guards/:id/deactivate	Guards page	Admin
GET	/admin/audit-log	Audit Log page	Admin

6 — Quick Commands

Copy these into your terminal whenever you need them.

Download the project	`git clone`
Switch to the dev branch	`git checkout dev`
Get the latest changes	`git pull origin dev`
Create your branch	`git checkout -b feature/`
Update your branch from dev	`git fetch origin then git rebase origin/dev`
Save a specific file	`git add path/to/file.tsx`
Write a commit	`git commit -m "feat: what you built"`
Send your branch to GitHub	`git push origin feature/`
Save unfinished work temporarily	`git stash`
Restore saved work	`git stash pop`
Check what has changed	`git status`
See your past commits	`git log --oneline`
Cancel a rebase that went wrong	`git rebase --abort`
Start the frontend	`cd frontend then npm run dev`
Start the backend	`cd backend then python run.py`
Start the database	`docker-compose up -d`
Stop the database	`docker-compose down`
Apply database changes	`flask db migrate -m 'message' then flask db upgrade`

Never commit your .env file. It holds passwords and secret keys. It is blocked by .gitignore, but double-check before you push.

Keep your commits small. One thing per commit makes it easy to find problems later and makes Pull Requests much faster to review.